



37. Softmax Regression

Softmax: When one class just isn't enough

Previous Section:

 [36. Decision Tree Regressor](#)

Contents:

[1. Softmax Regression – The Overview](#)

[2. Softmax Regression – The Iris Dataset](#)

[3. Softmax Regression – The Comprehensive Workflow](#)

[4. Softmax Regression with Python](#)

[Summary](#)

[References](#)

Softmax Regression, the name kinda speaks to you what it does. If you know what Logistic Regression is, and what a softmax function is.

Softmax Regression, like its predecessor, is a classification model, so do not let the name trick you (*well, I do not think you will, Logistic Regression has done that part for you, hoping that you learn your lesson*).

But now, let's be honest for a second. Logistic Regression had a very comfortable life. Two classes: Yes or No, 0 or 1 → Done. Clean. Simple. Binary.

But the real world? It rarely gives you that luxury. What if it's not just spam or not spam? What if it's: spam, promotion, social, updates? What if it's not just cat vs dog? But cat, dog, horse, bird... and maybe something you've never even seen before? Now suddenly, that "one output probability" from Logistic Regression feels insufficient. And this is exactly where Softmax Regression steps in.

Instead of asking: "What is the probability of this being class 1?". It asks a more powerful question: *"What is the probability of this belonging to each possible class?"*. Not one answer. But a distribution of answers. It doesn't just say: "This is class A." → It says: "There's a 70% chance it's A, 20% B, 10% C."

And that small shift? That changes everything. Because now, the model is no longer thinking in binary decision, it's thinking in competition. Each class is competing for probability. And the Softmax function makes sure of one important rule: **All probabilities must sum to 1** → Only one can dominate but all must be considered

So if Logistic Regression was about making a choice, Softmax Regression is about understanding the entire landscape before choosing. And that is what makes it powerful.

1. Softmax Regression – The Overview

As the introduction concludes: Softmax Regression doesn't come out of nowhere. It is not a completely new idea. It is simply what happens when Logistic Regression grows up and realizes the world is not just binary. Because Logistic Regression lives in a very controlled environment: Two classes → One probability → One decision

But the moment you introduce a third class, everything breaks. You can't just stretch the sigmoid function and hope it works. You need something that can handle multiple possibilities at the same time. And that's exactly what Softmax Regression does. Instead of predicting a single probability like: "What is the probability this belongs to class 1?" → It asks a broader question: "What is the probability this belongs to each possible class?". Not one answer, but a full distribution.

Now here's how it actually works under the hood. Just like Logistic Regression, we still start with a linear model. Each class gets its own score—what we call a *logit*. Think of logits as raw, unfiltered opinions of the model: *"Class A looks like this score"* → *"Class B looks like that score"* → *"Class C... something else"*.

But these scores are not probabilities. They can be anything: negative, positive, large, small... completely unbounded. So we need a way to convert them into something meaningful. And this is where the Softmax function comes in.

Instead of compressing a single value like sigmoid, Softmax transforms *all logits together* into probabilities. And it does something very clever: It

exponentiates each score → Then normalizes them by the total sum. So larger scores grow *much faster*, and smaller ones shrink toward zero.

The result? Every output is between 0 and 1 → All outputs sum to exactly 1. Which means we now have a proper probability distribution.

Right at the heart of this transformation is the Softmax formula:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Where: z_i is the raw score (logit) for class i ; k is the total number of classes

Now take a step back and think about what just happened. We started with raw scores, meaningless on their own. We applied a transformation that forces them to compete. And in the end, we get a clean probability distribution.

This is why Softmax Regression is so widely used. Whenever you see a model that needs to choose between multiple classes: Image classification; Text classification; Neural Networks. Especially in Deep Learning, Softmax almost always sits at the final layer, turning raw outputs into something interpretable.

So if Logistic Regression is about making a binary decision, Softmax Regression is about understanding *all possible choices*, and then deciding which one wins.

2. Softmax Regression – The Iris Dataset

Now let's make this real. To truly understand Softmax Regression, we need a dataset that actually *demand*s multiple classes.

And there is one dataset that has followed us around long enough. The famous **Iris dataset** (*and yes, I ended up bearing this name within my heart. Does an AI like me even have a heart? Let's not answer that right now.*)

Here's what the dataset looks like:

sepal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
7.0	3.2	4.7	1.4	Versicolor

sepal.length	sepal.width	petal.length	petal.width	variety
6.4	3.2	4.5	1.5	Versicolor
6.9	3.1	4.9	1.5	Versicolor
5.5	2.3	4.0	1.3	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.8	2.7	5.1	1.9	Virginica
7.1	3.0	5.9	2.1	Virginica
6.3	2.9	5.6	1.8	Virginica

Now each sample becomes a probability target over 3 classes.

Model Parameters – What Are We Learning? Instead of one weight vector like Logistic Regression, now we need one *per class*.

For each class k :

$$w_k = (w_{k0}, w_{k1}, w_{k2}, w_{k3}, w_{k4})$$

Stack them together, and we get:

$$W = \begin{bmatrix} w_{10} & w_{11} & w_{12} & w_{13} & w_{14} \\ w_{20} & w_{21} & w_{22} & w_{23} & w_{24} \\ w_{30} & w_{31} & w_{32} & w_{33} & w_{34} \end{bmatrix}$$

So in total: $3 \times 5 = 15$ parameters

Linear Scores (Logits) – Raw Opinions

For any input x , each class produces a score: $z_k = w_k^T x$

Let's take the first sample: $x = (1, 5.1, 3.5, 1.4, 0.2)$

We compute:

$$z_1 = w_{10} + 5.1w_{11} + 3.5w_{12} + 1.4w_{13} + 0.2w_{14}$$

$$z_2 = w_{20} + 5.1w_{21} + 3.5w_{22} + 1.4w_{23} + 0.2w_{24}$$

$$z_3 = w_{30} + 5.1w_{31} + 3.5w_{32} + 1.4w_{33} + 0.2w_{34}$$

These values $z_1, z_2, z_3, \dots$, are not probabilities. They are just *raw scores*. Unfiltered. Unbounded. Competing.

Softmax – Turning Competition into Probabilities

Now we take those logits, and force them to behave.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

For our 3 classes:

$$P(y = \text{Setosa} \mid x) = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$P(y = \text{Versicolor} \mid x) = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$P(y = \text{Virginica} \mid x) = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

And here's the rule Softmax enforces—no exceptions: $P_1 + P_2 + P_3 = 1$. This means the model doesn't just guess. It distributes belief across all classes

So now, instead of saying: "This is Setosa." → The model says something like: **"There's a 92% chance it's Setosa, 6% Versicolor, and 2% Virginica"**. And that is a completely different level of understanding.

Learning the Weights

Next, we'll move into the most important part: How the model actually *learns* these weights.

We arrive at the part where things stop being elegant—and start being *real*. Because unlike Linear Regression, Softmax Regression does not give you a clean, beautiful, closed-form solution. No shortcut. No direct equation that magically spits out the weights. Instead, it does something much more honest. It learns.

Think about what we are trying to do. We want the model to assign high probabilities to the correct class, and low probabilities to everything else. So we need a way to *measure* how good—or how bad—our predictions are. And this is where the idea of **loss** comes in.

From Likelihood to Loss - We could approach this from probability: ***“Maximize the likelihood of the correct labels.”*** → But in practice, we flip it into something easier to work with: Minimize the **cross-entropy loss**

This loss function penalizes the model when it is confident and wrong. And rewards it when it assigns high probability to the correct class.

So the objective becomes very clear: Find the weights W that minimize the loss, or equivalently, maximize the likelihood

No Closed Form → We Iterate

Since we cannot solve this directly, we improve the model step by step. Iteration by iteration. Adjustment by adjustment. And the tool we use for this? **Gradient Descent.**

Gradient Descent – From Simple to Complex

Back in the previous section, we introduced the basic idea:

$$x_{n+1} = x_n - \alpha f'(x)$$

A simple rule: Look at the slope (the gradient) → Move in the opposite direction
→ Repeat until things stop improving

But now, things are no longer that simple. We are not optimizing a single value anymore. We are optimizing an entire matrix of parameters.

Updating the Weight Matrix

So the update rule evolves into:

$$W_{new} = W_{old} - \alpha \nabla J(W)$$

Where: $J(W)$ is the cross-entropy loss; $\nabla J(W)$ is how the loss changes with respect to each weight; α is the learning rate (how big each step is).

Now pause for a second, and really think about what this means.

At every step: The model makes predictions. It compares them to reality. It measures how wrong it is → And then it *adjusts itself*. Not randomly. But guided by the gradient.

So in the end, Softmax Regression is not just a formula. It is a process. A loop that looks like this:

Predict → Measure error → Adjust weights → Repeat

Over and over again, until the model stabilizes.

And this is the key shift. Logistic Regression *felt* like solving an equation. Softmax Regression? It feels like training something. Because at its core, it is an **optimization-based probabilistic model**, designed to learn how to distribute belief across multiple classes

In the next section, we will bring everything together into a full workflow. And that's where you'll finally see the entire system moving as one

3. Softmax Regression – The Comprehensive Workflow

Now we stop talking *about* Softmax Regression, and we actually **walk through it**. Step by step. No skipping. No magic.

Step 1: Initialize the Weights

When doing this by hand, we keep things simple. We start with **zeros**:

$$W = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Structure:

- Rows = number of classes $K = 3$;
- Columns = number of features $d = 5$ (including intercept)

So: $W \in \mathbb{R}^{K \times d}$

At this point, the model knows nothing. Completely neutral. Completely ignorant.

Step 2: Compute the Logits

Take the first sample: $x = (1, 5.1, 3.5, 1.4, 0.2)$

The linear score (logit) for each class: $z_k = w_k^T x$

Since all weights are zero: $z_1 = 0, \quad z_2 = 0, \quad z_3 = 0$

Step 3: Apply the Softmax Function: $P_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$

Since all logits are equal:

$$P = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right)$$

The model is completely uncertain. Every class is equally likely.

Step 4: Compute the Loss

We use **cross-entropy loss**:

$$J = - \sum_{k=1}^K y_k \log(P_k)$$

For the first sample: True label: Setosa $\rightarrow y = (1, 0, 0)$

Step 5: Compute the Gradient

We need: $\frac{\partial J}{\partial w_k} \rightarrow$ Using the chain rule:

$$z_k = w_k^T x \Rightarrow \frac{\partial z_k}{\partial w_k} = x$$

And the key result (this is the beauty of Softmax + Cross-Entropy):

$$\frac{\partial J}{\partial w_k} = (P_k - y_k)x$$

Apply to the First Sample: $P = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right), \quad y = (1, 0, 0)$

Gradients:

- Class 1: $\left(\frac{1}{3} - 1 \right) x = -\frac{2}{3}x$
- Class 2: $\left(\frac{1}{3} - 0 \right) x = \frac{1}{3}x$
- Class 3: $\left(\frac{1}{3} - 0 \right) x = \frac{1}{3}x$

Step 6: Update the Weights (Gradient Descent)

Update rule: $w_k := w_k - \alpha \nabla w_k \rightarrow$ Let: $\alpha = 0.1$:

- Class 1: $w_1 = 0 - 0.1 \left(-\frac{2}{3}x \right) = 0.0667x$
- Class 2: $w_2 = 0 - 0.1 \left(\frac{1}{3}x \right) = -0.0333x$
- Class 3: $w_3 = -0.0333x$

Substitute $x = (1, 5.1, 3.5, 1.4, 0.2)$:

$$w_1 = (0.0667, 0.340, 0.233, 0.093, 0.013)$$

$$w_2 = (-0.033, -0.170, -0.117, -0.047, -0.007)$$

$$w_3 = (-0.033, -0.170, -0.117, -0.047, -0.007)$$

Combine them:

$$W = \begin{bmatrix} 0.0667 & 0.340 & 0.233 & 0.093 & 0.013 \\ -0.033 & -0.170 & -0.117 & -0.047 & -0.007 \\ -0.033 & -0.170 & -0.117 & -0.047 & -0.007 \end{bmatrix}$$

Step 7: Recompute Logits

- $z_1 = 0.0667 + 5.1(0.340) + 3.5(0.233) + 1.4(0.093) + 0.2(0.013) = 2.749$
- $z_2 = -0.033 + 5.1(-0.170) + 3.5(-0.117) + 1.4(-0.047) + 0.2(-0.007) = -1.3767$
- $z_3 = -0.033 + 5.1(-0.170) + 3.5(-0.117) + 1.4(-0.047) + 0.2(-0.007) = -1.3767$

Step 8: Apply Softmax Again

$$P(\text{Setosa}|x) = \frac{e^{2.749}}{e^{2.749} + e^{-1.3767} + e^{-1.3767}} = 0.9684$$

$$P(\text{Versicolor}|x) = \frac{e^{-1.3767}}{e^{2.749} + e^{-1.3767} + e^{-1.3767}} = 0.0156$$

$$P(\text{Virginica}|x) = \frac{e^{-1.3767}}{e^{2.749} + e^{-1.3767} + e^{-1.3767}} = 0.0156$$

After **just one update**. From complete uncertainty: $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}) \rightarrow$ To extreme confidence: $(0.9684, 0.0156, 0.0156)$

That's not random. That's the gradient doing exactly what it should: Increase the correct class $\rightarrow$ Decrease the incorrect ones

Another Iteration – Stochastic Gradient Descent (SGD)

Now we move to Sample 2: $x = (1, 4.9, 3.0, 1.4, 0.2)$, $y = (1, 0, 0)$

Previous prediction: $P = (0.9684, 0.0156, 0.0156)$

Gradient:

- Class 1: $(0.9684 - 1)x = -0.0316x$
- Class 2: $0.0156x$
- Class 3: $0.0156x$

Update again (same rule): $w_k := w_k - \alpha \nabla w_k$

Parameter changes after substituting x :

- $\Delta w_1 = (0.00316; 0.01548; 0.00948; 0.00442; 0.00063)$
- $\Delta w_2 = (-0.00156; -0.00764; -0.00468; -0.00218; -0.00031)$
- $\Delta w_3 = (-0.00156; -0.00764; -0.00468; -0.00218; -0.00031)$

Update the Weights

- $w_1 = (0.0667, 0.340, 0.233, 0.093, 0.013) + \Delta w_1$
- $w_2 = (-0.033, -0.170, -0.117, -0.047, -0.007) + \Delta w_2$
- $w_3 = (-0.033, -0.170, -0.117, -0.047, -0.007) + \Delta w_3$

Resulting matrix:

$$W = \begin{bmatrix} 0.0699 & 0.3555 & 0.2425 & 0.0974 & 0.0136 \\ -0.0346 & -0.1776 & -0.1217 & -0.0492 & -0.0073 \\ -0.0346 & -0.1776 & -0.1217 & -0.0492 & -0.0073 \end{bmatrix}$$

Recompute the Logits

- $z_1 = 0.0699 + 4.9(0.3555) + 3.0(0.2425) + 1.4(0.0974) + 0.2(0.0136) = 2.678$
- $z_2 = -0.0346 + 4.9(-0.1776) + 3.0(-0.1217) + 1.4(-0.0492) + 0.2(-0.0073) = -1.34$
- $z_3 = -0.0346 + 4.9(-0.1776) + 3.0(-0.1217) + 1.4(-0.0492) + 0.2(-0.0073) = -1.34$

New probabilities:

$$P(\text{Setosa}|x) = \frac{e^{2.678}}{e^{2.678} + e^{-1.34} + e^{-1.34}} = 0.9653$$

$$P(\text{Versicolor}|x) = \frac{e^{-1.34}}{e^{2.678} + e^{-1.34} + e^{-1.34}} = 0.01736$$

$$P(\text{Virginica}|x) = \frac{e^{-1.34}}{e^{2.678} + e^{-1.34} + e^{-1.34}} = 0.01736$$

→ Still very confident about Setosa for Sample 2.

What we just did is **Stochastic Gradient Descent**. Key idea: Update weights **after each sample**. Do NOT wait for the whole dataset.

Training Flow

- Epoch = one full pass through data
- If: $N = 12$, epochs = 100 → Total updates: $100 \times 12 = 1200$
- **The Loop**
 - Epoch 1: update after each sample
 - Epoch 2: continue from previous weights
 - Epoch 3: continue again
 - ...
 - Until convergence



Critical Insight (Do NOT Miss This): We never reset $\vec{W}$. Ever.

The model evolves continuously: Each sample nudges it. Each epoch refines it. Over time, it stabilizes

Softmax Regression does not “solve” a problem. It **learns** through small updates, repeated exposure, and gradual correction. And at the center of it all, is one simple force: $(P - y)x$. Quietly, consistently, pushing the model toward the correct answer.

4. Softmax Regression with Python

Softmax Regression is straightforward in Python using the Scikit-Learn library:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load iris dataset
X = load_iris().data
y = load_iris().target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Train the Logistic Regression model
# In Scikit-Learn, the softmax function is used in Logistic Regression when the target is multi-class
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

model1 = LogisticRegression()
model1.fit(X_train, y_train)
y_pred1 = model1.predict(X_test)
print(f"Accuracy Score: {accuracy_score(y_test, y_pred1)}")
```

Accuracy Score: 0.9333333333333333

```

probs = model1.predict_proba(X_test[:3])
pred_classes = model1.predict(X_test[:3])
print("\nPredicted Probabilities for First 3 Test Samples:")
for i in range(3):
    print(f"\nSample {i + 1}:")
    print(f"True Label: {y_test[i]}; Predicted Class: {pred_classes[i]}")
    print("Class Probabilities:", {int(c): float(p) for c, p
    in zip(model1.classes_, probs[i])})

```

Predicted Probabilities for First 3 Test Samples:

Sample 1:

True Label: 0; Predicted Class: 0

Class Probabilities: {0: 0.9723738400999122, 1: 0.02762612374574572, 2: 3.6154342025523304e-08}

Sample 2:

True Label: 0; Predicted Class: 0

Class Probabilities: {0: 0.9452580640271607, 1: 0.054741289536197, 2: 6.464366421454736e-07}

Sample 3:

True Label: 1; Predicted Class: 1

Class Probabilities: {0: 0.03149011995689202, 1: 0.9483340383327117, 2: 0.02017584171039624}

- Optionally, we can train a **Stochastic Gradient Descent model**

```
# Train a Stochastic Gradient Descent model
# max_iter in this case is number of epochs
from sklearn.linear_model import SGDClassifier

model2 = SGDClassifier(loss='log_loss', learning_rate='constant', eta0=0.1, max_iter=100, random_state=42)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)
print(f"Accuracy Score: {accuracy_score(y_test, y_pred2)}")
```

Accuracy Score: 0.7333333333333333

In Scikit-Learn, Softmax Regression can be implemented using two models:

- **LogisticRegression**: Uses the true Softmax formulation with cross-entropy loss. Optimized using advanced methods like L-BFGS. Batch optimization (stable, fast convergence) → Best for small to medium datasets
- **SGDClassifier (loss = 'log_loss')**: Also implements Softmax (multinomial logistic regression). Uses Stochastic Gradient Descent (SGD) and updates weights incrementally over epochs → More sensitive to learning rate, scaling, and tuning

Performance on Iris Dataset (Key Insight)

- **LogisticRegression**: 93-100% accuracy
- **SGDClassifier**: ~76.7% accuracy

Reason: Batch optimization (L-BFGS) is more precise and converges fully. SGD is approximate and may not fully converge without tuning.

Both models use the **same Softmax objective function**. Difference lies in *how they optimize it*. Optimization method can significantly affect performance. Small datasets → Batch methods perform better; Large datasets → SGD is more scalable

Summary

Softmax Regression is the natural extension of Logistic Regression to multi-class problems. Converts logits into a probability distribution Uses cross-

entropy loss for learning. Trained via optimization (typically Gradient Descent or its variants).

It introduces key ideas that appear throughout Machine Learning and Deep Learning: Probabilistic outputs; Gradient-based learning; Epochs and iterative updates

In many ways, Softmax Regression is the first real step into the world of Deep Learning.

References

<https://www.geeksforgeeks.org/python/softmax-regression-using-tensorflow/>

https://d2l.ai/chapter_linear-classification/softmax-regression.html

https://rasbt.github.io/mlxtend/user_guide/classifier/SoftmaxRegression/

<https://www.youtube.com/watch?v=KpKog-L9veg&t=261s>

Next Section:

 [38. Elastic Net Regression](#)